HTCondor Cluster Health & `run_as_owner` Diagnostic Playbooks

Scope: Windows CM, Windows Server 2022 execute nodes, RHEL 10 execute nodes, Windows workstation submit nodes. Mixed pool with `condor_credd` on a Windows Server node. Tests proceed from isolated subsystems to full end-to-end functional verification.

Conventions used throughout:

- `[WIN-CMD]` — run in an elevated (`Run as Administrator`) Command Prompt
- `[WIN-PS]` — run in an elevated PowerShell session
- `[RHEL]` — run as root or via sudo on RHEL 10
- `[ANY]` — can be run from any node in the pool
- Variables like `<CREDD_HOST>`, `<CM_HOST>` etc. are placeholders — substitute real values
- Collected output should be saved; many later playbooks reference earlier results

---

Playbook 0 — Environment Baseline

Run first on every node type. Establishes ground truth before testing anything else.

0.1 — HTCondor version and platform

cmd

```
:: [WIN-CMD] on CM, execute nodes, submit nodes
condor_version
```

Expected: all nodes report the same version string (25.0.x). Any mismatch should be noted — version skew between CM and nodes can cause subtle auth failures.

bash

```
# [RHEL] on all Linux execute nodes
condor_version
rpm -qi condor | grep -E "Version|Release|Architecture"
```

0.2 — Dump effective configuration to a file for comparison

cmd

```
:: [WIN-CMD] — run on EACH node type, save output
condor_config_val -dump > C:\Temp\condor_config_dump_%COMPUTERNAME%.txt 2>&1
```

bash

```
# [RHEL]
condor_config_val -dump > /tmp/condor_config_dump_$(hostname).txt 2>&1
```

Cross-check these files for the following variables — they MUST be identical across all nodes unless intentionally per-node:

```
CONDOR_HOST
COLLECTOR_HOST
UID_DOMAIN
FILESYSTEM_DOMAIN
CREDD_HOST
TRUST_UID_DOMAIN
SEC_DEFAULT_AUTHENTICATION
SEC_DEFAULT_AUTHENTICATION_METHODS
SEC_DAEMON_AUTHENTICATION_METHODS
ALLOW_READ
ALLOW_WRITE
ALLOW_DAEMON
ALLOW_NEGOTIATOR
```

Quick extraction:

cmd

```
:: [WIN-CMD]
for %v in (CONDOR_HOST COLLECTOR_HOST UID_DOMAIN FILESYSTEM_DOMAIN CREDD_HOST TRUST_
    echo %v = && condor_config_val %v
)
```

bash

```
# [RHEL]
for v in CONDOR_HOST COLLECTOR_HOST UID_DOMAIN FILESYSTEM_DOMAIN \
         CREDD_HOST TRUST_UID_DOMAIN; do
    printf "%s = %s\n" "$v" "$(condor_config_val $v)"
done
```

0.3 — Identify config file sources (detect overrides)

cmd

```
:: [WIN-CMD]
condor_config_val -config
```

This lists every file contributing to the effective configuration, in parse order. Verify the expected drop-in files appear and no unexpected files are present.

0.4 — Check which Windows account the condor service runs as

powershell

```
# [WIN-PS] — run on CM, credd host, execute nodes, submit nodes
Get-WmiObject Win32_Service -Filter "Name='condor'" |
    Select-Object Name, State, StartName, PathName |
    Format-List

# Also check the process token directly
$p = Get-Process condor_master -ErrorAction SilentlyContinue
if ($p) {
    & whoami /all | Select-String "User Name|SID"
}
```

Record the `StartName` value. This is the identity that daemon-to-daemon NTSSPI connections will present. It needs to appear in `ALLOW_DAEMON` on every receiving node.

Playbook 1 — Daemon Health

1.1 — Verify all expected daemons are running

cmd

```
:: [WIN-CMD] — CM (should show MASTER COLLECTOR NEGOTIATOR CREDD)
condor_config_val DAEMON_LIST
condor_status -any -direct <CM_HOST> 2>&1

:: Windows execute node (should show MASTER STARTD)
condor_config_val DAEMON_LIST
condor_status -any -direct <EXEC_HOST> 2>&1

:: Windows submit node (should show MASTER SCHEDD)
condor_config_val DAEMON_LIST
condor_status -any -direct <SUBMIT_HOST> 2>&1
```

bash

```
# [RHEL] — Linux execute node (should show MASTER STARTD)
condor_config_val DAEMON_LIST
systemctl status condor
condor_status -any -direct $(hostname -f) 2>&1
```

1.2 — Check daemon process list

powershell

```
# [WIN-PS]
Get-Process condor_* | Select-Object Name, Id, CPU, StartTime | Sort-Object Name
```

bash

```
# [RHEL]
ps aux | grep condor_ | grep -v grep
```

Expected processes per role:

Node role	Expected processes
CM	condor_master, condor_collector, condor_negotiator, condor_credd
Win execute	condor_master, condor_startd, condor_starter (when job running)
Win submit	condor_master, condor_schedd
RHEL execute	condor_master, condor_startd

1.3 — Check for recent daemon restarts or crashes in logs

powershell

```powershell
# [WIN-PS] — check for ERROR or STARTING UP lines in the last 100 lines of each log
$logdir = condor_config_val LOG
foreach ($log in Get-ChildItem "$logdir\*Log" -ErrorAction SilentlyContinue) {
    $last = Get-Content $log.FullName -Tail 100
    $hits = $last | Select-String "STARTING UP|EXITING|ERROR|EXCEPTION"
    if ($hits) {
        Write-Host "=== $($log.Name) ===" -ForegroundColor Yellow
        $hits
    }
}
```

bash

```bash
# [RHEL]
logdir=$(condor_config_val LOG)
for log in "$logdir"/*Log; do
    hits=$(tail -100 "$log" 2>/dev/null | grep -E "STARTING UP|EXITING|ERROR|EXCEPTI
    if [ -n "$hits" ]; then
        echo "=== $(basename $log) ==="
        echo "$hits"
    fi
done
```

1.4 — Verify credd daemon is listed in collector

cmd

```cmd
:: [ANY WIN-CMD]
condor_status -any | findstr /i "credd"
```

If `condor_credd` does not appear here, it is not advertising to the collector. The startd on execute nodes cannot locate it, which directly causes `LocalCredd = UNDEF`.

Playbook 2 — Network and DNS

DNS failures are the silent killer in HTCondor pools. Every subsequent test depends on this working correctly.

2.1 — Forward and reverse DNS for all pool nodes

Run from every node type targeting every other node type:

powershell

```powershell
# [WIN-PS] — test all pool hostnames
$hosts = @("<CM_HOST>", "<CREDD_HOST>", "<EXEC_WIN_01>", "<EXEC_RHEL_01>", "<SUBMIT_
foreach ($h in $hosts) {
    try {
        $fwd = [System.Net.Dns]::GetHostAddresses($h)
        foreach ($ip in $fwd) {
            $rev = [System.Net.Dns]::GetHostEntry($ip.ToString())
            Write-Host "FWD: $h -> $($ip.ToString())   REV: $($ip.ToString()) -> $($
        }
    } catch {
        Write-Host "FAIL: $h -- $_" -ForegroundColor Red
    }
}
```

bash

```bash
# [RHEL]
HOSTS=("<CM_HOST>" "<CREDD_HOST>" "<EXEC_WIN_01>" "<EXEC_RHEL_01>" "<SUBMIT_01>")
for h in "${HOSTS[@]}"; do
    ip=$(dig +short "$h" 2>/dev/null | tail -1)
    rev=$(dig +short -x "$ip" 2>/dev/null | sed 's/\.$//')
    printf "FWD: %-30s -> %-15s   REV: %-15s -> %s\n" "$h" "$ip" "$ip" "$rev"
done
```

Pass criteria: Forward and reverse DNS must agree on the FQDN for every node. HTCondor uses reverse DNS to verify identity claims. Mismatches cause auth failures that are difficult to trace.

2.2 — Verify HTCondor's own hostname resolution

HTCondor determines its hostname independently of the OS. Check what it resolves:

cmd

```cmd
:: [WIN-CMD]
condor_config_val FULL_HOSTNAME
condor_config_val IP_ADDRESS
```

bash

```
# [RHEL]
condor_config_val FULL_HOSTNAME
condor_config_val IP_ADDRESS
```

The `FULL_HOSTNAME` value must be the FQDN that other nodes can resolve. If it returns just a short hostname or the wrong FQDN, set `NETWORK_HOSTNAME` explicitly:

ini

```
NETWORK_HOSTNAME = correct-fqdn.example.com
```

2.3 — Verify UID_DOMAIN matches across all nodes

cmd

```
:: [ANY WIN-CMD or RHEL] — from CM, query all nodes at once
condor_status -af Machine UID_DOMAIN | sort
```

Every line must show the same `UID_DOMAIN` value. Any node showing a different value will fail job matching and credential lookups.

2.4 — TCP port 9618 reachability

powershell

```
# [WIN-PS] — test from submit node to CM, credd host, and execute nodes
$targets = @("<CM_HOST>", "<CREDD_HOST>", "<EXEC_WIN_01>")
foreach ($t in $targets) {
    $result = Test-NetConnection -ComputerName $t -Port 9618 -WarningAction Silentl
    $status = if ($result.TcpTestSucceeded) { "OPEN" } else { "BLOCKED" }
    Write-Host "$t :9618 -> $status"
}
```

bash

```
# [RHEL]
for t in <CM_HOST> <CREDD_HOST> <EXEC_WIN_01>; do
    if nc -z -w3 "$t" 9618 2>/dev/null; then
        echo "$t :9618 -> OPEN"
    else
        echo "$t :9618 -> BLOCKED"
    fi
done
```

If port 9618 is blocked between any pair of nodes, HTCondor will fail silently or with confusing error messages. Ensure Windows Firewall and any network ACLs permit TCP 9618 bidirectionally among all pool members.

Playbook 3 — Pool Signing Key / Pool Password Verification

These must be correct before any authentication tests will succeed.

3.1 — Verify the POOL signing key file exists on all Windows nodes

powershell

```powershell
# [WIN-PS] — run on CM, credd host, all execute nodes, all submit nodes
$keydir = condor_config_val SEC_PASSWORD_DIRECTORY
if (-not $keydir) { $keydir = "C:\ProgramData\HTCondor\passwords.d" }
$keyfile = Join-Path $keydir "POOL"

if (Test-Path $keyfile) {
    $fi = Get-Item $keyfile
    $acl = Get-Acl $keyfile
    Write-Host "EXISTS: $keyfile"
    Write-Host "  Size:     $($fi.Length) bytes"
    Write-Host "  Modified: $($fi.LastWriteTime)"
    Write-Host "  Owner:    $($acl.Owner)"
    # Show SHA256 of file for cross-node comparison
    $hash = Get-FileHash $keyfile -Algorithm SHA256
    Write-Host "  SHA256:   $($hash.Hash)"
} else {
    Write-Host "MISSING: $keyfile" -ForegroundColor Red
}
```

Critical check: Run this on every Windows node and compare the SHA256 hashes. They must all be identical. A node with a different or missing POOL file cannot validate IDTOKENS from the CM and cannot use the PASSWORD authentication method correctly.

3.2 — Verify the POOL signing key on RHEL nodes

bash

```
# [RHEL]
keydir=$(condor_config_val SEC_PASSWORD_DIRECTORY 2>/dev/null || echo "/etc/condor/p
keyfile="$keydir/POOL"

if [ -f "$keyfile" ]; then
    echo "EXISTS: $keyfile"
    echo "  Size:     $(stat -c%s $keyfile) bytes"
    echo "  Modified: $(stat -c%y $keyfile)"
    echo "  Owner:    $(stat -c%U:%G $keyfile)"
    echo "  Perms:    $(stat -c%a $keyfile)"
    echo "  SHA256:   $(sha256sum $keyfile | awk '{print $1}')"
    # Permissions must be 600 or the daemon will refuse to use it
    perms=$(stat -c%a $keyfile)
    if [ "$perms" != "600" ]; then
        echo "  WARNING: permissions should be 600, got $perms" >&2
    fi
else
    echo "MISSING: $keyfile" >&2
fi
```

3.3 — Verify the pool password is stored for condor_pool identity

cmd

```
:: [WIN-CMD on credd host] — as Administrator
condor_store_cred query -c
```

Expected output: `Credential is stored for condor_pool@<UID_DOMAIN>`

If it says "no credential" or returns an error, re-store it:

cmd

```
condor_store_cred add -c
:: Enter the pool password when prompted
```

3.4 — Verify IDTOKEN infrastructure

cmd

```
:: [WIN-CMD on CM]
condor_token_list
```

```bash
# [RHEL CM]
condor_token_list
```

Check that daemon tokens exist in `C:\ProgramData\HTCondor\tokens.d\` (Windows) or `/etc/condor/tokens.d/` (RHEL) on each non-CM node:

```powershell
# [WIN-PS] — on execute/submit nodes
$tokendir = "C:\ProgramData\HTCondor\tokens.d"
if (Test-Path $tokendir) {
    Get-ChildItem $tokendir | ForEach-Object {
        Write-Host "Token file: $($_.Name)  Size: $($_.Length)"
    }
} else {
    Write-Host "Token directory missing: $tokendir" -ForegroundColor Red
}
```

Playbook 4 — Authentication Tests (`condor_ping` Progression)

Work through authorization levels from weakest to strongest. Each successive level requires more trust. Stop at the first failure and investigate before continuing — later levels will not pass if earlier ones fail.

4.1 — Enable security debugging for all ping tests

Set this environment variable in your shell before running any `condor_ping` commands:

```cmd
:: [WIN-CMD]
set _condor_SEC_TOOL_DEBUG=D_SECURITY:2
```

```bash
# [RHEL]
export _condor_SEC_TOOL_DEBUG=D_SECURITY:2
```

4.2 — Local self-ping (baseline)

cmd

```
:: [WIN-CMD on each node — targets local master]
condor_ping -verbose DAEMON CONFIG READ WRITE
```

This must succeed everywhere with no auth errors. If a node can't ping itself, the local HTCondor installation is broken before any networking is involved.

4.3 — CM → execute node pings

cmd

```
:: [WIN-CMD on CM] — test all authorization levels against each execute node
for %h in (<EXEC_WIN_01> <EXEC_WIN_02> <EXEC_RHEL_01> <EXEC_RHEL_02>) do (
    echo === Testing %h ===
    condor_ping -verbose -name %h -type STARTD READ WRITE DAEMON CONFIG
    echo.
)
```

4.4 — Execute node → CM pings

cmd

```
:: [WIN-CMD on each execute node]
condor_ping -verbose -type COLLECTOR READ WRITE DAEMON
condor_ping -verbose -type NEGOTIATOR READ WRITE DAEMON NEGOTIATOR
```

4.5 — Submit node → CM pings

cmd

```
:: [WIN-CMD on submit node]
condor_ping -verbose -type COLLECTOR READ WRITE
condor_ping -verbose -type SCHEDD READ WRITE DAEMON
```

4.6 — Critical: all Windows nodes → credd host

cmd

```
:: [WIN-CMD on each execute node and submit node]
condor_ping -verbose -name <CREDD_HOST> -type CREDD READ WRITE DAEMON
```

DAEMON must succeed here. If it fails, `LocalCredd` will never populate.

4.7 — Capture and compare `condor_ping` identity strings

cmd

```
:: [WIN-CMD] — run on each node, record the "Identity" column for DAEMON level
condor_ping -verbose -name <CREDD_HOST> -type CREDD DAEMON 2>&1 | findstr /i "identi
```

Create a table:

Source node	Target	Identity presented	DAEMON decision
CM	credd	?	?
EXEC_WIN_01	credd	?	?
SUBMIT_01	credd	?	?
EXEC_RHEL_01	credd	?	?

The identity in the DAEMON row must match what is in `CREDD.ALLOW_DAEMON` on the credd host. If it doesn't, that is your authorization mismatch.

Playbook 5 — Authorization Configuration Verification

5.1 — Dump and verify all ALLOW_* lists on all nodes

cmd

```
:: [WIN-CMD] — run on each node
for %v in (ALLOW_READ ALLOW_WRITE ALLOW_DAEMON ALLOW_ADMINISTRATOR ALLOW_NEGOTIATOR
    echo %v:
    condor_config_val %v
    echo.
)
```

bash

```
# [RHEL]
for v in ALLOW_READ ALLOW_WRITE ALLOW_DAEMON ALLOW_ADMINISTRATOR \
         ALLOW_NEGOTIATOR ALLOW_CONFIG ALLOW_ADVERTISE_STARTD \
         ALLOW_ADVERTISE_SCHEDD; do
    printf "%s:\n  %s\n\n" "$v" "$(condor_config_val $v)"
done
```

5.2 — Verify credd-specific authorization

cmd

```
:: [WIN-CMD on credd host]
for %v in (CREDD.ALLOW_DAEMON CREDD.ALLOW_WRITE CREDD.ALLOW_READ CREDD.SEC_DAEMON_AU
    echo %v:
    condor_config_val %v
    echo.
)
```

Expected values for a correctly configured credd:

```
CREDD.ALLOW_DAEMON              = condor_pool@EXAMPLE.COM
CREDD.SEC_DAEMON_AUTHENTICATION_METHODS = PASSWORD
CREDD.SEC_DEFAULT_AUTHENTICATION = REQUIRED
CREDD.SEC_DEFAULT_ENCRYPTION    = REQUIRED
CREDD.SEC_DEFAULT_INTEGRITY     = REQUIRED
```

5.3 — Verify PASSWORD is in daemon auth methods on all Windows nodes

cmd

```
:: [WIN-CMD] — run on credd host, all execute nodes, all submit nodes
condor_config_val SEC_DAEMON_AUTHENTICATION_METHODS
```

The word `PASSWORD` must appear in the output. If it is absent, daemon-to-daemon connections will never authenticate as `condor_pool@...`, and the credd will reject them.

5.4 — Check that STARTER_ALLOW_RUNAS_OWNER is set on execute nodes

cmd

```
:: [WIN-CMD on each Windows execute node]
condor_config_val STARTER_ALLOW_RUNAS_OWNER
```

Must return `True`. If False or undefined, run_as_owner jobs will silently fall back to the slot user.

5.5 — Check CREDD_CACHE_LOCALLY

cmd

```
:: [WIN-CMD on all Windows nodes except credd host itself]
condor_config_val CREDD_CACHE_LOCALLY
condor_config_val CREDD_HOST
```

`CREDD_CACHE_LOCALLY` must be `True` and `CREDD_HOST` must resolve to the correct hostname.

Playbook 6 — Credd Subsystem Isolation Tests

These tests isolate the credd daemon independently from job submission.

6.1 — Verify credd is listening and reachable

cmd

```
:: [WIN-CMD on credd host]
condor_status -any | findstr /i credd

:: From a remote node:
condor_ping -verbose -name <CREDD_HOST> -type CREDD READ WRITE DAEMON
```

6.2 — Turn up credd logging to maximum

Add to credd host config temporarily:

ini

```
CREDD_DEBUG     = D_FULLDEBUG D_SECURITY:2 D_COMMAND
MAX_CREDD_LOG   = 200000000
```

cmd

```
condor_reconfig
```

Leave this in place for all subsequent credd tests. The CreddLog will now show every connection attempt, auth method tried, identity established, and credential fetch request.

6.3 — Attempt pool password store from each node type

Run from each Windows node type in sequence, watching CreddLog in real time:

```cmd
:: [WIN-CMD — new window, on credd host, watch log]
powershell -Command "Get-Content 'C:\ProgramData\HTCondor\log\CreddLog' -Wait -Tail
```

Then from the CM, execute node, and submit node:

```cmd
:: [WIN-CMD — on each source node]
condor_store_cred add -c -debug
:: Enter the pool password
```

For each attempt, CreddLog should show:

```
PERMISSION GRANTED to condor_pool@EXAMPLE.COM from <source-ip>
```

If you see `PERMISSION DENIED`, note the identity string in the log line — that is what you need to add to `CREDD.ALLOW_DAEMON`.

6.4 — Query pool credential from each node

```cmd
:: [WIN-CMD on each node]
condor_store_cred query -c -debug
```

Expected: `Credential is stored for condor_pool@EXAMPLE.COM`

If this fails from some nodes and not others, compare the `ALLOW_CONFIG` and `ALLOW_WRITE` settings — `condor_store_cred query` uses CONFIG-level access for the initial connection to the schedd.

6.5 — Test user credential storage

For each test user, from their own submit node session:

```cmd
:: [WIN-CMD — as the test user, not as Administrator]
condor_store_cred query -debug
```

Then store it:

cmd

```
condor_store_cred add -debug
:: Enter the user's Windows domain password
```

Then query again to confirm it was stored:

cmd

```
condor_store_cred query -debug
```

Check CreddLog on the credd host — it should show the user's credential being stored:

```
PERMISSION GRANTED to <USERNAME>@EXAMPLE.COM from <source-ip>
```

Playbook 7 — LocalCredd Population Tests

`LocalCredd` appearing in the startd ClassAd is the gating signal for `run_as_owner` job matching. This playbook verifies and forces its population.

7.1 — Check current LocalCredd status across all execute nodes

cmd

```
:: [ANY WIN-CMD or RHEL]
condor_status -af Machine LocalCredd | sort
```

Each Windows execute node should show the credd hostname. RHEL nodes will show nothing (LocalCredd is a Windows-only attribute). If any Windows execute node shows blank, `LocalCredd` is not set on that node.

More detailed per-slot view:

cmd

```
condor_status -f "%-30s" Name -f "%-30s" Machine -f "%s\n" ifThenElse(isUndefined(Lo
```

7.2 — Check HasWindowsRunAsOwner

cmd

```
condor_status -af Machine HasWindowsRunAsOwner
```

For Windows execute nodes, this must be `true`. If it is `false` or undefined, `STARTER_ALLOW_RUNAS_OWNER` is not set, or the startd ClassAd has not refreshed.

7.3 — Force a startd ClassAd refresh

If LocalCredd is UNDEF despite the credd being reachable:

cmd

```
:: [WIN-CMD on CM]
condor_reconfig -all
```

Wait 60 seconds. The startd on each execute node will re-run its credd probe and update its ClassAd.

cmd

```
:: Wait, then check again
ping -n 60 localhost > nul
condor_status -f "%-30s" Machine -f "%s\n" ifThenElse(isUndefined(LocalCredd),"UNDEF
```

7.4 — Manually probe credd connectivity from an execute node's StartLog

On a Windows execute node where LocalCredd is UNDEF, add to config:

ini

```
STARTD_DEBUG = D_FULLDEBUG D_SECURITY:2
```

Reconfig, then check StartLog for lines containing `credd` or `LocalCredd`:

powershell

```
# [WIN-PS on execute node]
$log = condor_config_val LOG
Get-Content "$log\StartLog" | Select-String -Pattern "credd|LocalCredd|credential" -
```

Look for lines like:

```
Contacting CREDD at <ip>:port
Successfully contacted CREDD: set LocalCredd to <hostname>
```

or failure lines:

```
Failed to contact CREDD at <ip>: <reason>
```

Playbook 8 — UID_DOMAIN and Identity Consistency

8.1 — Verify UID_DOMAIN is consistent across the pool

cmd

```
:: [ANY WIN-CMD]
condor_status -af Machine UID_DOMAIN | sort -u
```

Must return exactly one unique value. Multiple values indicate misconfiguration.

8.2 — Verify what identity a job will be recorded under

Submit a test probe job and check what Owner attribute it receives:

Create `probe.sub`:

```
executable = C:\Windows\System32\cmd.exe
arguments  = /C whoami > C:\Temp\whoami_test.txt
output     = C:\Temp\probe.out
error      = C:\Temp\probe.err
log        = C:\Temp\probe.log
queue
```

cmd

```
condor_submit probe.sub
condor_q -format "%s\n" Owner
```

Note the Owner format. It must be `username` (not `DOMAIN\username`). HTCondor stores the job owner without the domain prefix; the domain is inferred from `UID_DOMAIN`. If the owner shows `DOMAIN\username`, the UID_DOMAIN matching is broken.

8.3 — Verify credential is stored under the correct identity

cmd

```
:: [WIN-CMD on submit node — as the test user]
condor_store_cred query -debug 2>&1 | findstr /i "account\|credential\|user"
```

The account name reported must match the format `username@UID_DOMAIN`. If it shows `DOMAIN\username@UID_DOMAIN` (double-qualified), the credd lookup will fail.

8.4 — Cross-check via condor_store_cred query with explicit username

cmd

```
:: [WIN-CMD on credd host as Administrator]
condor_store_cred query -u testuser@EXAMPLE.COM
```

If this fails but `condor_store_cred query -u testuser` succeeds, there is a domain name format mismatch. Ensure `UID_DOMAIN` equals the domain portion of what Windows presents.

Playbook 9 — Job Matching Verification

Before testing actual execution, verify the ClassAd requirements will match.

9.1 — Verify `HasWindowsRunAsOwner` and `LocalCredd` requirements match

cmd

```
:: [WIN-CMD on submit node]
condor_status -constraint "HasWindowsRunAsOwner == true && isString(LocalCredd)" \
    -af Machine LocalCredd
```

This lists all execute nodes that are eligible for `run_as_owner` jobs. If empty, no jobs with `run_as_owner = True` will ever be matched.

9.2 — Run condor_q -better-analyze on a held or idle run_as_owner job

If a job is already stuck idle or on hold:

cmd

```
condor_q -better-analyze <JOBID>
```

Look for the `TARGET.HasWindowsRunAsOwner` and `TARGET.LocalCredd` conditions in the output. These show which requirement is failing and how many machines satisfy each condition.

9.3 — Simulate a run_as_owner requirements expression

cmd

```
:: [WIN-CMD] — count machines that would match a run_as_owner job
condor_status -constraint "(OpSys == \"WINDOWS\") && HasWindowsRunAsOwner && isStrin
    -af Machine HasWindowsRunAsOwner LocalCredd
```

Playbook 10 — Functional Job Tests (Build-Up Sequence)

Each test builds on the previous. Do not proceed to the next test until the current one passes cleanly.

Test 10.1 — Baseline: plain job to Windows execute node, no run_as_owner

Creates `test01_baseline.sub`:

```
# test01_baseline.sub
# Simplest possible Windows job. No run_as_owner. Verifies basic job flow.
executable   = C:\Windows\System32\cmd.exe
arguments    = /C echo Baseline OK > C:\Temp\condor_test01.txt
output       = C:\Temp\test01.out
error        = C:\Temp\test01.err
log          = C:\Temp\test01.log
requirements = (OpSys == "WINDOWS")
queue
```

cmd

```
condor_submit test01_baseline.sub
condor_watch_q
```

Pass criteria:

- Job leaves queue without going on hold
- `C:\Temp\condor_test01.txt` exists on the execute node containing `Baseline OK`
- No errors in `test01.log`

Test 10.2 — Baseline: plain job to RHEL execute node

Creates `test02_linux.sub`:

```
# test02_linux.sub
executable   = /bin/hostname
output       = /tmp/condor_test02.out
error        = /tmp/condor_test02.err
log          = /tmp/condor_test02.log
requirements = (OpSys == "LINUX")
queue
```

bash

```
condor_submit test02_linux.sub
condor_watch_q
```

Test 10.3 — run_as_owner: verify identity in job output

Creates `test03_whoami.sub`. The job writes `whoami` output to a local path so we can confirm it ran as the correct user:

```
# test03_whoami.sub
executable   = C:\Windows\System32\cmd.exe
arguments    = /C whoami /all > C:\Temp\condor_test03_whoami.txt
output       = C:\Temp\test03.out
error        = C:\Temp\test03.err
log          = C:\Temp\test03.log
requirements = (OpSys == "WINDOWS")
run_as_owner = True
queue
```

cmd

```
condor_submit test03_whoami.sub
condor_watch_q
```

Pass criteria:

- Job completes without going on hold
- `C:\Temp\condor_test03_whoami.txt` on the execute node contains the **submitting user's domain username**, not a slot user (e.g. `condor-slot1` or similar)

If the job goes on hold, retrieve the hold reason:

cmd

```
condor_q -hold -format "%-12d" ClusterId -format "%-12d" ProcId -format "%s\n" HoldR
```

Test 10.4 — run_as_owner: UNC path read access

This test verifies that credentials are being impersonated correctly — the job reads from a UNC share that only the submitting user can access.

Pre-requisite: ensure `\\<FILESERVER>\<SHARE>\condor_test_input.txt` exists and is readable only by the test user (not by the machine account or slot users).

Creates `test04_unc_read.sub`:

```
# test04_unc_read.sub
executable   = C:\Windows\System32\cmd.exe
arguments    = /C type \\<FILESERVER>\<SHARE>\condor_test_input.txt >
C:\Temp\test04_output.txt
output       = C:\Temp\test04.out
error        = C:\Temp\test04.err
log          = C:\Temp\test04.log
requirements = (OpSys == "WINDOWS")
run_as_owner = True
queue
```

Pass criteria:

- Job completes without hold
- `C:\Temp\test04_output.txt` on execute node contains the content of the input file
- No `Access Denied` or `Network path not found` in error output

If this fails but test 10.3 passed (identity was correct), the issue is Kerberos ticket delegation or SMB session token propagation — confirm the file server grants access to the user identity seen in test 10.3.

Test 10.5 — run_as_owner: UNC path write access

Creates `test05_unc_write.sub`:

```
# test05_unc_write.sub
# Output and log files go directly to the UNC share
executable   = C:\Windows\System32\cmd.exe
arguments    = /C echo Write test OK
output       = \\<FILESERVER>\<SHARE>\condor_test05.out
error        = \\<FILESERVER>\<SHARE>\condor_test05.err
log          = \\<FILESERVER>\<SHARE>\condor_test05.log
requirements = (OpSys == "WINDOWS")
run_as_owner = True
queue
```

Pass criteria:

- Job completes without hold
- Output file appears on the UNC share with correct content
- Log file appears on the UNC share

Test 10.6 — run_as_owner: multi-slot stress test

Submit 20 concurrent run_as_owner jobs to stress the credential caching:

cmd

```
:: Create test06_stress.sub
(
echo executable   = C:\Windows\System32\cmd.exe
echo arguments    = /C whoami
echo output       = C:\Temp\test06_$(Process).out
echo error        = C:\Temp\test06_$(Process).err
echo log          = C:\Temp\test06.log
echo requirements = (OpSys == "WINDOWS"^)
echo run_as_owner = True
echo queue 20
) > test06_stress.sub

condor_submit test06_stress.sub
condor_watch_q
```

Pass criteria:

- All 20 jobs complete without holds
- None run as slot user (spot-check a few output files)

- CreddLog on credd host shows successful credential fetches, no auth failures

Test 10.7 — run_as_owner: job targeting both Windows and RHEL (platform switching)

This verifies that `run_as_owner = True` in a submit file does not prevent the job from also running on RHEL (where it should be ignored gracefully):

```
# test07_any_platform.sub
executable   = /usr/bin/id
# On Windows this would be: executable = C:\Windows\System32\cmd.exe
# This test uses Linux exec nodes only
output       = /tmp/test07.out
error        = /tmp/test07.err
log          = /tmp/test07.log
requirements = (OpSys == "LINUX")
run_as_owner = True
queue
```

bash

```
condor_submit test07_any_platform.sub
condor_watch_q
```

Pass criteria:

- Job runs on a RHEL execute node without error
- The `run_as_owner` attribute is silently ignored on Linux (not held)

Playbook 11 — Log Analysis Scripts

11.1 — Unified PERMISSION DENIED scanner

Run after any failed test to gather all denial events:

powershell

```powershell
# [WIN-PS] — scan all daemon logs for denials
$logdir = & condor_config_val LOG
$pattern = "PERMISSION DENIED|DC_AUTHENTICATE.*failed|SECMAN.*error|authentication.*
Get-ChildItem "$logdir\*Log" | ForEach-Object {
    $matches = Select-String -Path $_.FullName -Pattern $pattern -CaseSensitive:$fal
    if ($matches) {
        Write-Host "`n=== $($_.Name) ===" -ForegroundColor Red
        $matches | ForEach-Object { Write-Host $_.Line }
    }
}
```

bash

```bash
# [RHEL]
logdir=$(condor_config_val LOG)
pattern="PERMISSION DENIED|DC_AUTHENTICATE.*failed|SECMAN.*error|authentication.*fai
for log in "$logdir"/*Log; do
    hits=$(grep -iE "$pattern" "$log" 2>/dev/null)
    if [ -n "$hits" ]; then
        echo -e "\n=== $(basename $log) ==="
        echo "$hits"
    fi
done
```

11.2 — credd credential fetch scanner

After submitting run_as_owner jobs, confirm the credd served credentials:

powershell

```powershell
# [WIN-PS on credd host]
$logdir = & condor_config_val LOG
$creddlog = "$logdir\CreddLog"
Write-Host "=== Credential fetches ==="
Select-String -Path $creddlog -Pattern "credential for user|PERMISSION GRANTED|PERMI
    Select-Object -Last 50 |
    ForEach-Object { Write-Host $_.Line }
```

11.3 — LocalCredd timeline from StartLog

powershell

```
# [WIN-PS on execute node]
$logdir = & condor_config_val LOG
Select-String -Path "$logdir\StartLog" -Pattern "LocalCredd|credd|credential" |
    ForEach-Object { Write-Host $_.Line }
```

Playbook 12 — Comprehensive Health Summary Script

Run this script from the CM to produce a go/no-go summary across the entire pool. Save as `C:\Temp\htcondor_health_check.ps1` on the CM:

powershell

```powershell
# htcondor_health_check.ps1
# Run from the CM as Administrator.
# Produces a pass/fail summary for all major subsystems.

$errors = @()
$warnings = @()
$uidDomain = & condor_config_val UID_DOMAIN

function Check($label, $expr, $expected, $actual) {
    if ($actual -match [regex]::Escape($expected)) {
        Write-Host "[PASS] $label" -ForegroundColor Green
    } else {
        Write-Host "[FAIL] $label" -ForegroundColor Red
        Write-Host "       Expected: $expected"
        Write-Host "       Got:      $actual"
        $script:errors += $label
    }
}

Write-Host "`n====== HTCondor Health Check ======`n"

# --- Daemon checks ---
Write-Host "-- Daemon Health --"
$anyOutput = & condor_status -any 2>&1
Check "Collector has entries" "MyType" "MyType" ($anyOutput -join "`n")
Check "CREDD advertised" "credd" "credd" ($anyOutput -join "`n")

# --- UID_DOMAIN consistency ---
Write-Host "`n-- UID_DOMAIN Consistency --"
$domains = & condor_status -af Machine UID_DOMAIN 2>&1 | Sort-Object -Unique
if ($domains.Count -eq 1) {
    Write-Host "[PASS] UID_DOMAIN consistent: $($domains[0])" -ForegroundColor Green
} else {
    Write-Host "[FAIL] UID_DOMAIN inconsistent across nodes:" -ForegroundColor Red
    $domains | ForEach-Object { Write-Host "       $_" }
    $errors += "UID_DOMAIN inconsistency"
}

# --- HasWindowsRunAsOwner ---
Write-Host "`n-- run_as_owner Readiness --"
$runas = & condor_status -constraint "(OpSys == `"WINDOWS`") && HasWindowsRunAsOwner
if ($runas) {
    Write-Host "[PASS] Windows execute nodes ready for run_as_owner:" -ForegroundCol
```

```
    $runas | ForEach-Object { Write-Host "       $_" }
} else {
    Write-Host "[FAIL] No Windows execute nodes are ready for run_as_owner" -Foregro
    $errors += "No run_as_owner capable nodes"
}

# --- LocalCredd UNDEF check ---
$undefNodes = & condor_status -constraint "(OpSys == `"WINDOWS`") && isUndefined(Loc
if ($undefNodes) {
    Write-Host "[WARN] Windows nodes with LocalCredd UNDEF:" -ForegroundColor Yellow
    $undefNodes | ForEach-Object { Write-Host "       $_" }
    $warnings += "Some nodes have LocalCredd UNDEF"
}

# --- Pool password ---
Write-Host "`n-- Pool Password --"
$credq = & condor_store_cred query -c 2>&1
if ($credq -match "Credential is stored") {
    Write-Host "[PASS] Pool password stored: $credq" -ForegroundColor Green
} else {
    Write-Host "[FAIL] Pool password NOT stored on this node" -ForegroundColor Red
    $errors += "Pool password not stored on CM"
}

# --- STARTER_ALLOW_RUNAS_OWNER ---
Write-Host "`n-- STARTER_ALLOW_RUNAS_OWNER (local check) --"
$sarow = & condor_config_val STARTER_ALLOW_RUNAS_OWNER 2>&1
Check "STARTER_ALLOW_RUNAS_OWNER" "True" "True" $sarow

# --- condor_ping DAEMON to credd host ---
Write-Host "`n-- condor_ping DAEMON to credd host --"
$creddHost = & condor_config_val CREDD_HOST
$pingResult = & condor_ping -verbose -name $creddHost -type CREDD DAEMON 2>&1
if ($pingResult -match "ALLOW") {
    Write-Host "[PASS] DAEMON ping to credd host succeeded" -ForegroundColor Green
} else {
    Write-Host "[FAIL] DAEMON ping to credd host failed" -ForegroundColor Red
    $errors += "DAEMON ping to credd host failed"
}

# --- Summary ---
Write-Host "`n====== Summary ======"
if ($errors.Count -eq 0 -and $warnings.Count -eq 0) {
    Write-Host "ALL CHECKS PASSED" -ForegroundColor Green
```

```
} else {
    if ($errors.Count -gt 0) {
        Write-Host "ERRORS ($($errors.Count)):" -ForegroundColor Red
        $errors | ForEach-Object { Write-Host "  - $_" }
    }
    if ($warnings.Count -gt 0) {
        Write-Host "WARNINGS ($($warnings.Count)):" -ForegroundColor Yellow
        $warnings | ForEach-Object { Write-Host "  - $_" }
    }
}
```

Run it:

```
powershell -ExecutionPolicy Bypass -File C:\Temp\htcondor_health_check.ps1
```

Quick Reference — Failure Symptom Lookup

Symptom	First playbook to check
`LocalCredd = UNDEF` on all Windows exec nodes	PB 6 (credd isolation), PB 3.3 (pool password)
`LocalCredd = UNDEF` on some nodes only	PB 2 (DNS), PB 4.6 (DAEMON ping to credd)
`condor_ping DAEMON` fails, CONFIG passes	PB 5.2 (CREDD.ALLOW_DAEMON), PB 4.2 (identity)
`condor_store_cred query` fails with ALLOW_WRITE error	PB 5.1 (ALLOW_CONFIG), PB 2.4 (port 9618)
Job goes on hold: `Could not locate valid credential`	PB 8 (UID_DOMAIN / identity format)
Job goes on hold: `Failed to initialize user_priv as (null)\username`	PB 8.3 (NTDomain format), PB 0.4 (service account)
Job runs as slot user instead of owner	PB 5.4 (STARTER_ALLOW_RUNAS_OWNER), PB 7.2 (HasWindowsRunAsOwner)
RHEL execute nodes not appearing in pool	PB 3.2 (POOL key), PB 2.1 (DNS)

Symptom	First playbook to check
`condor_status` shows no output	PB 2.4 (port 9618), PB 1.4 (collector)
Jobs idle with 0 machines matched	PB 9.1 (ClassAd requirements), PB 0.2 (UID_DOMAIN)